

UNIVERSIDAD TECNOLÓGICA DE TEHUACÁN

TECNOLOGÍAS DE LA INFORMACIÓN DESARROLLO DE SOFTWARE

EJEMPLO INTEGRADOR S7

COMUNICACIÓN ASÍNCRONA ACCESIBLE

PRESENTA:

OSCAR ESPINOZA LANDETA

8 “A”

A CARGO:

ING. JOSÉ MIGUEL CARRERA PACHECO



1. Async JavaScript: El Motor bajo el Capó

Para dominar la asincronía en React, no basta con saber usar `async/await`; hay que entender el **Event Loop** para evitar bloqueos en el hilo principal (*Main Thread*).

Microtasks vs. Macrotasks

JavaScript prioriza las tareas. Las promesas (incluyendo `fetch`) generan **Microtasks**. Estas tienen prioridad absoluta: el navegador no renderizará nada nuevo hasta que la cola de Microtasks esté vacía.

- **Peligro:** Un bucle infinito de microtareas congelará la UI de tu app de React, aunque técnicamente no sea un "bloqueo" del sistema operativo.

La Anatomía de una Race Condition en React

Ocurre frecuentemente en buscadores o filtros. Si el usuario teclea rápido:

1. Petición A (palabra "hola") se dispara.
2. Petición B (palabra "hola mundo") se dispara.
3. La Petición B llega en 100ms. La Petición A llega en 500ms (por latencia).
4. **Resultado:** La UI muestra los resultados de "hola", aunque el input diga "hola mundo".

Solución Técnica: El patrón `ignore`.

```
JavaScript
useEffect(() => {
  let ignore = false;
  async function startFetching() {
    const json = await fetchResults(query);
    if (!ignore) { setData(json); }
  }
  startFetching();
  return () => { ignore = true; }; // Función de limpieza
}, [query]);
```

AbortController

No es solo para ahorrar ancho de banda; es para salud de memoria. Al usar `controller.abort()`, la promesa del `fetch` se rechaza inmediatamente con un `AbortError`, permitiendo que el bloque `catch` limpie el estado.

2. Consumo de APIs REST: Semántica y Resiliencia

El frontend es el primer validador de la integridad de los datos. No debemos confiar ciegamente en lo que el backend envía.

Semántica HTTP Avanzada

Método	Idempotente	Seguro	Uso en React
GET	Sí	Sí	Obtener datos (no debe tener efectos secundarios).
POST	No	No	Crear recursos (siempre genera un ID nuevo).
PUT	Sí	No	Reemplazar un recurso completo.
PATCH	No	No	Modificación parcial (ahorra ancho de banda).
DELETE	Sí	No	Eliminar (si borras dos veces, el resultado es el mismo: no existe).

Estrategia de Reintentos (Retry Logic)

No todos los errores merecen un reintento.

- **Error 400 (Bad Request):** No reintentar (el cliente envió algo mal).
- **Error 401 (Unauthorized):** No reintentar (necesita refrescar token).
- **Error 503 (Service Unavailable): Reintentar con Exponential Backoff.**
 - *Fórmula de espera:* $\$delay = 2^n \times 100ms$ (donde n es el número de intento). Esto evita el efecto "manada" que tumba servidores.

3. Estados de Carga Accesibles (A11y)

La accesibilidad no es un "extra", es parte de la arquitectura. Un loader invisible para un lector de pantalla es un bug.

Atributos WAI-ARIA Críticos

1. **aria-busy="true"**: Se aplica al contenedor del contenido que está cambiando. Informa que el elemento está en transición.
2. **role="status"**: Ideal para mensajes de carga cortos. Es un "live region" con prioridad **polite** por defecto (no interrumpe al lector de pantalla, espera a que termine de leer).
3. **aria-live="polite" vs "assertive"**:

- **polite**: "Oye, cuando puedas, avisa que ya cargó".
- **assertive**: "¡Interrumpe todo! Los datos ya están aquí". (Usar solo para errores críticos).

Prevención de Layout Shift (CLS)

El uso de **Skeleton Loaders** es la mejor práctica técnica porque:

- Mantiene el árbol de nodos del DOM estable.
- Evita que el navegador tenga que recalcular la geometría de la página (Reflow) cuando llegan los datos, mejorando el rendimiento de renderizado.

4. Manejo de Errores: La Capa de Resiliencia

Un error no manejado es un "Silent Failure". Debemos diferenciar el origen del problema.

Clasificación de Fallos

1. **Error de Capa de Transporte (Network Error)**: El **fetch** falla antes de tocar el servidor (DNS, Firewall, modo avión). El objeto **response** ni siquiera existe.
2. **Error de Lógica de Negocio (Server Error)**: El servidor respondió (ej. 404).
Ojo: **fetch** no lanza error en un 404. Debes verificar manualmente **if (!response.ok)**.

Registro y Logging

En producción, no uses **console.log**. Implementa un **Error Boundary** en React para capturar errores de renderizado y usa un interceptor de red para loggear errores de API con metadatos:

- Endpoint fallido.
- User-Agent.
- Timestamp exacto.
- Request ID (si el backend lo proporciona).

Resumen de la Investigación para tu Proyecto

- **Async**: Usa **AbortController** en cada **useEffect** de carga.
- **API**: Implementa un **interceptor** (si usas Axios) o un **wrapper** de fetch para centralizar los Status Codes.
- **A11y**: Cambia los spinners genéricos por Skeletons con **aria-busy**.
- **Errores**: No permitas que la app muera; usa estados de "Fallback" amigables.